

MODULE 1 — Gestion de Projet Agile (Scrum & Kanban) + Rôle PO/SM

Durée recommandée : 3 jours (21 heures)

Niveau : Mixte (débutant à intermédiaire)

1. Objectifs de la formation

- Comprendre les fondamentaux de l'Agilité.
- Maîtriser les frameworks **Scrum** et **Kanban**.
- Savoir structurer un projet agile de bout en bout.
- Clarifier les rôles clés : **Product Owner** & **Scrum Master**.
- Savoir animer les cérémonies agiles et gérer un backlog.
- Utiliser des outils modernes : Jira, Trello, ClickUp, GitHub Projects.

2. Programme détaillé

Jour 1 — Introduction & Fundaments de l'Agilité

- L'origine de l'Agilité : historique & Manifeste Agile.
- Les 4 valeurs et 12 principes.
- Les approches prédictives vs adaptatives.
- Concepts clés :
 - MVP
 - Valeur métier
 - Feedback loop
 - Definition of Ready/Done
- Atelier pratique : Analyse d'un cas projet et transition vers l'Agile.

Jour 2 — Scrum : Cadre, rôles et cérémonies

- Les rôles :
 - **Product Owner** (responsabilités, outils, backlog, priorisation, story mapping).
 - **Scrum Master** (facilitateur, remove impediments, servant leadership).
 - **Equipe de développement** (autonomie, auto-organisation).
- Les artefacts Scrum :
 - Product Backlog
 - Sprint Backlog
 - Increment
- Les cérémonies :
 - Sprint Planning
 - Daily Scrum
 - Sprint Review

- Sprint Retrospective
- Techniques de découpage : User stories, Epics, Tasks.
- Atelier : Création d'un Product Backlog et d'un Sprint.

Jour 3 — Kanban & Avancée dans la gestion Agile

- Kanban : principes & valeurs.
- Limitation du WIP, visualisation, lead time, throughput.
- Tableaux Kanban : règles, optimisation des flux.
- Comparaison Scrum vs Kanban : quand utiliser quoi ?
- Gestion de projet hybride (ScrumBan).
- Atelier : Mise en place d'un tableau Kanban et simulation de flux.
- Outils recommandés : Jira, Trello, Asana, GitHub Projects.

3. Modalités pédagogiques

- Alternance **théorie / ateliers pratiques**.
- Études de cas réels.
- Exercices collaboratifs et simulations de cérémonies.
- Supports numériques fournis.

MODULE 2 — Culture et Outils DevOps (CI/CD, Docker, Kubernetes)

Durée recommandée : 4 jours (28 heures)

Niveau : Mixte (débutant à intermédiaire)

1. Objectifs de la formation

- Assimiler la culture et les piliers du DevOps.
- Mettre en place des pipelines **CI/CD**.
- Maîtriser les notions de conteneurisation (Docker).
- Déployer des applications sur **Kubernetes**.
- Utiliser des outils modernes : GitLab CI, GitHub Actions, Jenkins, Docker Hub, ArgoCD.

2. Programme détaillé

Jour 1 — Culture DevOps & CI/CD

- Origine et philosophie DevOps : CAMS, collaboration & automation.
- Cycle de vie DevOps : Plan, Code, Build, Test, Release, Deploy, Operate.
- Introduction à Git & Git Flow.
- Mise en place d'un pipeline CI simple (tests automatisés).
- Atelier : Pipeline CI sur GitHub Actions / GitLab CI.

Jour 2 — Docker : Conteneurisation

- Introduction à Docker : images, conteneurs, volumes, réseaux.
- Dockerfile : bonnes pratiques de construction.
- Docker Compose pour l'orchestration locale.
- Gestion des registres d'images (Docker Hub, GitHub Container Registry).
- Atelier : Conteneurisation d'une application (Node/Spring/Angular selon besoin).

Jour 3 — Introduction à Kubernetes

- Architecture Kubernetes : Pods, Deployments, Services, Ingress.
- Namespace, ConfigMaps, Secrets.
- RBAC & sécurité dans Kubernetes.
- Exposition des applications via Ingress.
- Atelier : Déploiement d'une application en cluster Kubernetes local (Minikube/K3s).

Jour 4 — Automation & DevOps avancé

- CI/CD complet avec build, test, analyse, déploiement automatique.
- Introduction à Helm Charts.
- Observabilité : Prometheus, Grafana (aperçu).
- Notions de GitOps (ArgoCD).
- Atelier final : Chaîne complète CI/CD → Docker → Kubernetes.

3. Modalités pédagogiques

- Démonstrations en direct (live coding).
- Ateliers sur des pipelines CI/CD réels.
- Travail en groupe sur un mini projet.
- Support PDF + commandes scripts fournis.

MODULE 3 — Framework Spring Security (5 jours)

Durée : 5 jours (35 heures)

Niveau : Débutant → Avancé

1. Objectifs pédagogiques

À l'issue de cette formation, les participants seront capables de :

- Comprendre les concepts modernes de sécurité applicative.
- Maîtriser **Spring Security 6+** et sa nouvelle architecture.
- Implémenter l'authentification et l'autorisation sur des APIs REST.
- Construire un système complet basé sur **JWT** (access/refresh tokens).
- Mettre en place **OAuth2**, **OpenID Connect**, **Keycloak**, et l'intégration avec un front (Angular/React).
- Sécuriser les endpoints en RBAC (roles & permissions).
- Appliquer les bonnes pratiques : CORS, CSRF, password policy, logging, monitoring.

2. Programme détaillé

Jour 1 — Fondamentaux de la sécurité & architecture Spring Security

A. Concepts fondamentaux

- Authentification vs autorisation
- Stateless vs stateful
- Password hashing (BCrypt, Argon2)
- Attaques courantes : brute force, MITM, XSS, CSRF

B. Architecture Spring Security 6

- SecurityFilterChain (nouveau modèle)
- AuthenticationManager & AuthenticationProvider
- UserDetails & GrantedAuthority
- Difference entre WebSecurityConfigurerAdapter (legacy) et nouvelle approche

C. Premier Hands-On

- Création d'un projet Spring Boot 3
- Mise en place d'un système d'authentification basique
- Configuration HTTPS / certificats autosignés (optionnel)

Jour 2 — Authentication avancée & Authorization

A. Authentication

- Form login / Basic Auth
- Custom UserDetailsService
- PasswordEncoder configuration
- Authentification multi-layer

B. Authorization (RBAC / Rules)

- Roles, Authorities & Permissions
- Méthodes d'autorisation :
 - @PreAuthorize
 - @Secured
 - Expression-Based Access Control
- Gestion fine des droits par ressource

C. Ateliers pratiques

- Mise en place d'un modèle Role / Permission
- Protection de endpoints REST selon profils

Jour 3 — JWT & Sécurité des APIs REST

A. Pourquoi JWT ?

- JWT vs Session JSESSIONID
- Structure d'un JWT (header, payload, signature)
- Access Token vs Refresh Token

B. Implémentation complète

- Création d'un JWTAuthenticationFilter
- Création d'un JWTAuthorizationFilter
- Gestion des erreurs (EntryPoint, AccessDeniedHandler)
- Rotation de tokens & refresh flow
- Stockage en base vs stateless total

C. Ateliers pratiques

- Mise en place d'une API REST sécurisée
- Ajout des endpoints : /auth/login, /auth/refresh, /users, /admin...
- Tests Postman & automatisation

Jour 4 — OAuth2, OpenID Connect & Keycloak

A. Concepts OAuth2 / OIDC

- Authorization Code Flow
- Implicit Flow
- PKCE
- Tokens OIDC (ID token vs access token)

B. Spring Security + OAuth2

- Client OAuth2 (Google, GitHub)
- Server Resource & Authorization Server (Spring Authorization Server ou Keycloak)

C. Keycloak

- Installation & configuration
- Clients, Realm, Roles, Claims
- Mappers, tokens, policies
- Sécurisation d'une API Spring Boot via Keycloak

D. Atelier

- Login OAuth2 avec Keycloak
- Protection d'un front Angular/React

Jour 5 — Sécurité avancée, bonnes pratiques & atelier final

A. Sécurisation réelle d'une application

- CORS vs CSRF
- Sécurisation des endpoints Websocket
- Rate limiting & throttling
- Exception handling
- Logging & audit (Spring Boot Actuator)

B. Stratégies avancées

- Sécurité des microservices
- Intégration API Gateway (Spring Cloud Gateway)
- JWT signing key rotation
- Hardening Spring Security (OWASP)

C. Atelier final (grand TP)

- Construction d'un système complet :
 - ✓ API REST sécurisée JWT
 - ✓ Console admin / endpoints RBAC
 - ✓ Intégration Keycloak (auth + roles)
 - ✓ Front-end protégé en OAuth2 (démon)
 - ✓ Documentation Postman + sécurité
- Évaluation & Q/A

3. Modalités pédagogiques

- Alternance théorie / pratique
- Mini-projet évolutif sur 5 jours
- Supports PDF, code source & exercices fournis
- Évaluation finale